BACHELOR OF COMPUTER SCIENCE
FACULTY/SCHOOL OF SCHOOL OF COMPUTER SCIENCE
BINA NUSANTARA UNIVERSITY
JAKARTA

ASSESSMENT FORM

Course: COMP6048001 - Data Structure

Method of Assessment: Case Study

Semester/Academic Year : 2/2024-2025

Name of Lecturer : YUDY PURNAMA, S.Kom., M.TI

Date : 19/05/2025

Class : LD09

Topic : Tries

Group Members :	1 Valentino Kurniawan - 2802524120 2_____ 3_____ 4_____ 5_____ 6_____ 7_____ 8_____

Student Outcomes:
SO 2 - Mampu merancang, mengimplementasikan, dan mengevaluasi solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi dalam konteks ilmu computer
Able to design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of computer science

Learning Objectives:
LObj 2.2 - Mampu mengimplementasikan solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi tertentu dalam konteks ilmu computer
Able to implement a computing-based solution to meet a given set of computing requirements in the context of computer science

Learning Outcomes:
LO 3 - Apply data structures using C

No	Related LO-LOBJ-SO	Assessment criteria	Weight	Excellent (85 - 100)	Good (75-84)	Average (65-74)	Poor (0 - 64)	Score	(Score x Weight)
1	LO 3 – LObj 2.2 – SO 2	Ability to design an algorithm for the problem	**80 %**	Able to solve 76 - 100% the problem with fully functional feature.	Able to solve 51-75% the problem and lack some features.	Able to solve 26-50% of the problem and lack some features.	Able to solve less than 25% of the problem.		
2	LO 3 – LObj 2.2 – SO 2	Ability to explain the algorithm and make a documentation	**20 %**	Able to explain the algorithm well. The documentation is good and complete.	Able to give a good explaination of the algorithm. The documentation is good enough.	Able to give an enough explaination of the algorithm The documentation is good enough, but it is not complete.	The explanation the algorithm is bad. The documentation is not complete.		
		Total Score: ∑(Score x Weight)							

Remarks:

_____

_____

_____

_____

ASSESSMENT METHOD

Instructions:

- This case study is individual project.
- The case study scoring consist of two parts:
  1. **[LO3-LObj 2.2-SO 2, 80 points]** Design an algorithm in **cpp** to solve the problem.
  2. **[LO3-LObj 2.2-SO 2, 20 points]** Create a documentation for your program in PDF (Explain the logic, purpose, and maybe limitation behind each line of the code/function with human language), **please write the code, don't just screenshot it**. Test your program with **custom cases that you made** and put the custom cases along with the result (**screenshots**) in the documentation PDF.
     - Example code explanation *may* look like this (**Explaining each line of code**):

```
include<stdio.h>  // included for standard input output function

int main(){
    // declare variable number as integer
    int number;
    // Ask for input number from 1 to 10. This code will loop while the number inputted is either less than 1 or more than 10.
    do{
        printf("Enter number between 1 & 10: ");
        scanf("%d", &number);
    }while(number < 1 || number > 10);
    return 0;
}
```

     - Example code explanation for function *may also* look like this (**Explaining how a function works**):

```
int biggerNumber(int a, int b){
    int bigger = (a > b) ? a : b;
    return big
}
```

The code above is used to find a bigger number from 2 numbers which are in the parameter: 'a' and 'b'. To find the bigger number, I simply use a ternary operator which has the condition if 'a' is more than 'b', then 'a' is the bigger number, else then 'b' is the bigger number, I assign either value in a temporary variable 'bigger', then return it. By using this logic, the function will return 'b' if the numbers have the same value, which is okay in our program.

- The custom case **(that you made yourself)** should **at least** have:
    - **Input of 15 slang words**,
    - **Search 5 words**,
    - **View prefix 5 words** and
    - **View all**.
- Submit the .**cpp** and **PDF** files to Binusmaya (zip them).

Note for Lecturers:

- The lecture notifies this case study to the student from Week 1.
- Deadline for the case study *ideally* is in week 9. However, the deadline set in Binusmaya will be in the last week.
- The student should submit the report to binusmaya no later than deadline.
- **If the students do plagiarism, their score for this case study will be 0 (zero). The lecturers have the privilage to determine whether the students do plagiarism.**

DOCUMENTATION

- The libraries used in this program:

```
#include <stdio.h> // Standard input output
#include <stdlib.h> // For malloc() and exit()
#include <string.h> // For strcpy()
#include <ctype.h> // To check character in the string
#include <stdbool.h> // Booleans (true or false) because C dont have boolean
#include <conio.h> // For _getch()
#include <windows.h> // For system()

#define ALPHABET_SIZE 68 // Defines alphabet, number, and special characters
// There are 26 alphabets (a-z case insensitive), 10 numbers (0-9), and 32 special characters
(!-~)
```

- The `Util` functions:

```
bool firstMenu = true;
// Flag to control whether to add delay (Sleep) when showing the title (only on first time).

void Title(){

   printf("  ==========================================================   \n\n");  if(firstMenu)
Sleep(200);
   printf("  BBBBB                                      LL              \n");   if(firstMenu)
Sleep(200);
   printf(" BB   BB                                     LL              \n");   if(firstMenu)
Sleep(200);
   printf(" BB   BB                                GG   LL              \n");   if(firstMenu)
Sleep(200);
   printf(" BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE     \n");   if(firstMenu)
Sleep(200);
   printf(" BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE    \n");   if(firstMenu)
Sleep(200);
   printf(" BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE   \n");   if(firstMenu)
Sleep(200);
   printf(" BB   BB    OO   OO    OO   OO    GG   GG  LL   EE         \n");   if(firstMenu)
Sleep(200);
   printf("  BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE     \n");   if(firstMenu)
Sleep(200);
   printf"                                     GG                   \n");   if(firstMenu)
Sleep(200);
   printf("                                     GG                   \n");   if(firstMenu)
Sleep(200);
   printf("                                GG   GG                   \n");   if(firstMenu)
Sleep(200);
   printf("                                 GGGGG                    \n\n"); if(firstMenu)
Sleep(200);
   printf("  ==========================================================   \n\n");  if(firstMenu)
Sleep(200);
// Draws the full art "Boogle" title with animation effect on first run.
```

```
        firstMenu = false;
        // Prevents the animation from repeating in future title displays.
    }
```

The Title() function prints an ASCII-art styled banner of the word "Boogle" to the console. On the first display, it adds a short animation effect using delays (Sleep(200)) between each line to enhance the visual presentation. After the first time, the animation is skipped for faster rendering in future calls.

```
    void Press_Any_Key_To_Continue(){

        printf(" >> Press any key to continue...");
        // Display prompt to inform the user.

        _getch();
        // Wait for the user to press any key (input is not shown on screen).

        system("cls");
        // Clear the screen after a key is pressed.
    }
```

The Press_Any_Key_To_Continue() function pauses the program's execution, waiting for the user to press any key. Once a key is pressed, the screen is cleared. This is typically used to give users time to read a message or result before returning to the main interface.

```
    void Hide_Cursor(){

        printf("\e[?25l");
        // ANSI escape code to hide the text cursor in the terminal.

        fflush(stdout);
        // Ensure the output is immediately flushed to the screen.
    }
```

The Hide_Cursor() function hides the blinking text cursor from the terminal using an ANSI escape sequence. This improves the visual experience, especially in custom menu systems or animations, by preventing cursor distractions. It's commonly used in user interfaces that take full control of the console display.

```
    void Show_Cursor(){
```

```
        printf("\e[?25h");
        // ANSI escape code to show the text cursor in the terminal.

        fflush(stdout);
        // Immediately flush the output buffer to ensure the cursor visibility change takes effect.
    }
```

The Show_Cursor() function makes the terminal’s blinking text cursor visible again using an ANSI escape code. It's typically used after previously hiding the cursor (e.g. with Hide_Cursor()), restoring the terminal to its normal behavior for text input or user interaction.

- The main program functions:

```
void Menu_Guide(){

   Title();  // Display the program title banner

   printf(" << ");
   const char* message1 = "Welcome to Boogle,\n";
   for(int i = 0; message1[i] != '\0'; i++){
         printf("%c", message1[i]);     // Print each character of message1
         Sleep(50);                     // Add typing effect delay
   }

   printf(" << ");
   const char* message2 = "A place to release and find slang words!\n";
   for(int i = 0; message2[i] != '\0'; i++){
         printf("%c", message2[i]);     // Print message2
         Sleep(50);
   }

   printf("\n");

   printf(" << ");
   const char* message3 = "Press 'w' or 's' to navigate the menu,\n";
   for(int i = 0; message3[i] != '\0'; i++){
         printf("%c", message3[i]);     // Print message3
         Sleep(50);
   }

   printf(" << ");
   const char* message4 = "Then 'Enter' to select!\n\n";
   for(int i = 0; message4[i] != '\0'; i++){
         printf("%c", message4[i]);     // Print message4
         Sleep(50);
   }

   Press_Any_Key_To_Continue();  // Wait for user before returning
```

```
}
```

The Menu_Guide() function introduces the user to the program ("Boogle") and explains how to interact with the main menu using keys 'w', 's', and 'Enter'. The messages are animated using a delay between characters (Sleep(50)), enhancing user engagement. This function is meant to be shown the first time the app runs or as a help screen.

```
char Menu(){

   char choice = '1';  // Default choice is option 1

   while(true){
         system("cls");  // Clear screen each loop to refresh display

         Title();  // Print title banner

         // Highlight the current selected menu option
         if(choice == '1') printf(" >> Release\n");
         else printf(" Release\n");

         if(choice == '2') printf(" >> Search\n");
         else printf(" Search\n");

         if(choice == '3') printf(" >> View with prefix\n");
         else printf(" View with prefix\n");

         if(choice == '4') printf(" >> View all\n");
         else printf(" View all\n");

         if(choice == '5') printf(" >> Exit\n");
         else printf(" Exit\n");

         // Detect keypress for navigation
         switch(_getch()){
               case 'W':           // Uppercase 'W'
               case 'w':           // Lowercase 'w'
                     choice -= 1;     // Move up
```

```
                        if(choice < '1') choice = '5';  // Wrap around to bottom
                        break;

                  case 'S':           // Uppercase 'S'
                  case 's':           // Lowercase 's'
                        choice += 1;    // Move down
                        if(choice > '5') choice = '1';  // Wrap around to top
                        break;

                  case '\r': return choice;  // Enter key pressed: return selected option
            }
      }

      return choice;
   }
```

The Menu() function displays an interactive **main menu** with 5 options: Release, Search, View with prefix, View all, Exit. Users navigate the menu using **'w' or 's'** keys (case-insensitive) and **select an option with 'Enter'**. The currently selected item is highlighted with >>. The screen is cleared and redrawn on each input for a clean UI experience.

```
   int main(){

      Hide_Cursor();
      // Hide the console cursor to make the UI look cleaner for the user.

      Menu_Guide();
      // Show the user guide or instructions for using the application.

      while(true){
            // Run an infinite loop so the menu keeps appearing after each action,
            // allowing the user to perform multiple operations without restarting the app.

            switch(Menu()){
                  // Call the Menu() function to get the user's input,
                  // then execute the appropriate case based on the input.
```

```
                case '1':
                      Release_New_Slang();
                      // Option 1: Add a new slang word to the dictionary.
                      break;

                case '2':
                      Search_Slang();
                      // Option 2: Search for an existing slang word by name.
                      break;

                case '3':
                      View_With_Prefix();
                      // Option 3: Let user input a prefix and show matching slang words.
                      break;

                case '4':
                      View_All();
                      // Option 4: Display the entire contents of the slang dictionary.
                      break;

                case '5':
                      Exit();
                      // Option 5: End the program.
                      break;
            }
      }

      return 0;
      // Standard return value for main, although it's never reached due to the infinite loop.
   }
```

The main() function controls the overall flow of the program by continuously displaying a menu and responding to the user's input. It allows users to add, search, and view slang words or exit the application. This loop ensures the program remains interactive until the user chooses to quit.

- Option 1 functions:

```
void Release_Guide(){

   Title();
   // Display the title header.

   printf(" << ");
   const char* message1 = "Release Guide :\n";
   for(int i = 0; message1[i] != '\0'; i++){
         printf("%c", message1[i]);
         Sleep(50);
   }
   // Print the first guide message character-by-character with a typing effect.

   printf(" << ");
   const char* message2 = "1. Release new slang into the dictionary!\n";
   for(int i = 0; message2[i] != '\0'; i++){
         printf("%c", message2[i]);
         Sleep(50);
   }
   // Second message with typing effect.

   printf(" << ");
   const char* message3 = "2. If slang exist already, you will change the description!\n\n";
   for(int i = 0; message3[i] != '\0'; i++){
         printf("%c", message3[i]);
         Sleep(50);
   }
   // Third message explaining what happens if the slang already exists.

   Press_Any_Key_To_Continue();
   // Wait for user input before clearing the screen and continuing.
}
```

The Release_Guide() function displays a short instructional guide on how to add or update slang in the dictionary. It uses a typing animation effect to show each message character by character, enhancing the user experience. After the guide is fully shown, it waits for the user to press a key before proceeding.

```
void Validate_Word(char* word){

   if(strlen(word) > 1) printf(" << Slang word more than 1 character [v]\n");
   else printf(" << Slang word more than 1 character [ ]\n");
   // Check if the slang word is longer than 1 character. Display a checkbox indicator with
[v] if valid.

   if(!strchr(word, ' ')) printf(" << Slang word contains no space [v]\n\n");
   else printf(" << Slang word contains no space [ ]\n\n");
   // Check if the slang word does not contain any space. Again, show [v] if valid.
}
```

The Validate_Word() function provides real-time visual feedback to the user when entering a slang word. It checks two conditions: (1) the word must be more than one character long, and (2) it must not contain spaces. It prints a validation checklist showing whether each requirement is currently satisfied using [v] for valid and [ ] for invalid.

```
void Validate_Description(char* space){

   if(space && (isalnum(space[1]) || ispunct(space[1])))
         printf(" << Description more than 1 word [v]\n\n");
   else printf(" << Description more than 1 word [ ]\n\n");
   // Checks if the description contains a space followed by a valid character,
   // implying there is more than one word. Displays a check indicator accordingly.
}
```

The Validate_Description() function checks if a slang description contains **more than one word** by verifying if there is a space followed by a valid character (alphanumeric or punctuation). It provides a visual checklist to guide the user, indicating whether this condition is met with [v] or not with [ ].

```
typedef struct Trie{

   Trie* alphaNumeric[ALPHABET_SIZE];
   // Array of pointers to child nodes, representing all possible alphanumeric and punctuation
characters.

   bool isWord;
```

```
      // Indicates whether this node marks the end of a complete slang word.

      char* description;
      // Stores the description of the slang word if isWord is true.

   } node;

   node* Create(){

      node* newNode = (node*)malloc(sizeof(node));
      // Allocate memory for a new Trie node.

      for(int i = 0; i < ALPHABET_SIZE; i++)
            newNode->alphaNumeric[i] = NULL;
      // Initialize all child pointers to NULL.

      newNode->isWord = false;
      // By default, the new node does not mark the end of a word.

      return newNode;
      // Return the newly created node.
   }

   node* ROOT = Create();
   // Initialize the root of the Trie with an empty node.
```

This code defines a Trie data structure (struct Trie) used to store slang words and their descriptions. Each node can point to multiple child nodes through the alphaNumeric array, representing possible characters in slang terms. The isWord flag marks the end of a valid slang word, and description stores the associated meaning. The Create() function dynamically allocates and initializes a new node. ROOT is the starting point of the Trie, from which all words are stored or searched.

```
   void Insert(node* curr, char* word, char* description){

      int character = 0;
      while(*word){
            // Determine the index in the alphaNumeric array based on character type
```

```
        if(isalpha(*word) && isupper(*word))
            character = *word - 'A';                    // A-Z → 0-25
        if(isalpha(*word) && islower(*word))
            character = *word - 'a';                    // a-z → 0-25 (same index range)
        if(isdigit(*word))
            character = *word - '0' + 26;               // 0-9 → 26-35
        if(ispunct(*word) && '!' <= *word && *word <= '/')
            character = *word - '!' + 36;               // '!' to '/' → 36-50
        if(ispunct(*word) && ':' <= *word && *word <= '@')
            character = *word - ':' + 51;               // ':' to '@' → 51-57
        if(ispunct(*word) && '[' <= *word && *word <= '`')
            character = *word - '[' + 58;               // '[' to '`' → 58-63
        if(ispunct(*word) && '{' <= *word && *word <= '~')
            character = *word - '{' + 64;               // '{' to '~' → 64+

        if(!curr->alphaNumeric[character])
            curr->alphaNumeric[character] = Create();   // Create node if it doesn't exist

        curr = curr->alphaNumeric[character];           // Move to the next node
        word++;
    }

    system("cls");
    Title();

    if(curr->isWord){
        // If the word already exists, update its description
        printf(" << The slang exist already in the dictionary,\n");
        printf(" << Description updated!\n\n");
        free(curr->description);
    } else{
        // If it's a new word, mark it and confirm addition
        printf(" << New slang has been released into the dictionary!\n\n");
        curr->isWord = true;
    }
```

```
        // Allocate memory for the description and copy it
        curr->description = (char*)malloc(strlen(description) * sizeof(char) + 1);
        strcpy(curr->description, description);

        Press_Any_Key_To_Continue(); // Prompt user before continuing
    }
```

The Insert() function adds a new slang word and its description into the Trie-based dictionary. It translates each character in the input word into an index for the alphaNumeric array, supporting letters, digits, and selected punctuation. If nodes for characters don't exist, it creates them. If the word already exists, its description is updated. Otherwise, it is inserted as a new entry. After insertion, the description is saved, and feedback is shown to the user.

```
    bool notGuided = true;
    // Flag to ensure the guide is only shown once when adding a new slang word.

    void Release_New_Slang(){

        system("cls");
        // Clear the console screen.

        if(notGuided){
            Release_Guide();
            notGuided = false;
            // Show guidance for using this feature only on the first time.
        }

        Show_Cursor();
        // Show the cursor for user input.

        int i = 0;
        char word[256] = {0};
        // Initialize index and buffer for the slang word input.

        while(true){
            // Loop to get valid slang word input from the user.
```

```
        system("cls");
        Title();
        // Clear screen and display title again.

        Validate_Word(word);
        // Validate the current word input.

        printf(" >> Input slang word ['Esc' to return to menu]: %s", word);
        // Prompt the user to input a slang word.

        char ch = _getch();
        // Get a single character input without echoing it.

        if(ch == '\e'){
            Hide_Cursor();
            return;
            // If Esc is pressed, hide cursor and exit.
        }

        if(ch == '\r'){
            if(strlen(word) > 1 && !strchr(word, ' ')) break;
            else continue;
            // If Enter is pressed, only continue if the word is longer than 1 char and has
no spaces.
        }

        if(ch == '\b' && i > 0) word[--i] = '\0';
        // Handle Backspace: remove last character if possible.

        if((isalnum(ch) || ispunct(ch) || isspace(ch)) && i < 255){
            word[i++] = ch;
            word[i] = '\0';
            // Add character to word if valid.
        }
    }
```

```
    i = 0;
    char description[256] = {0};
    // Reset index and create buffer for the description.

    while(true){
        // Loop to get a valid description from the user.

        system("cls");
        Title();
        // Clear and show title again.

        printf(" << Slang word: \"%s\"\n\n", word);
        // Display the word already entered.

        char* space = strchr(description, ' ');
        Validate_Description(space);
        // Check if there's a valid word after a space in the description.

        printf(" >> Input description ['Esc' to return to menu]: %s", description);
        // Prompt the user to input a description.

        char ch = _getch();
        // Read user input character without echoing.

        if(ch == '\e'){
            Hide_Cursor();
            return;
            // Exit if user presses Esc.
        }

        if(ch == '\r'){
            if(space && (isalnum(space[1]) || ispunct(space[1]))) break;
            else continue;
            // On Enter: ensure the description includes a space and valid character after
it.
        }
```

```
            if(ch == '\b' && i > 0) description[--i] = '\0';
            // Handle backspace for description.

            if((isalnum(ch) || ispunct(ch) || isspace(ch)) && i < 255){
                  description[i++] = ch;
                  description[i] = '\0';
                  // Append valid character to description.
            }
      }

      Hide_Cursor();
      // Hide cursor once input is finished.

      printf("\n\n");
      // Print spacing before inserting.

      Insert(ROOT, word, description);
      // Add the new slang word and its description to the dictionary/tree.

      return;
      // End the function.
   }
```

The Release_New_Slang() function allows users to add a new slang word along with its description to the dictionary. It ensures the inputs are valid, provides guidance on the first use, and supports user-friendly editing through character-by-character input. Once both the word and description are correctly entered, they are inserted into the data structure for later use.

- Option 2 functions:

```
node* Search(node* curr, char* word){

   int character = 0;
   while(*word){
         // Map the character to its respective index in the Trie

         if(isalpha(*word) && isupper(*word))
               character = *word - 'A';                  // A-Z → 0-25
         if(isalpha(*word) && islower(*word))
               character = *word - 'a';                  // a-z → 0-25
         if(isdigit(*word))
               character = *word - '0' + 26;             // 0-9 → 26-35
         if(ispunct(*word) && '!' <= *word && *word <= '/')
               character = *word - '!' + 36;             // Punctuation '!'-'/' → 36-50
         if(ispunct(*word) && ':' <= *word && *word <= '@')
               character = *word - ':' + 51;             // Punctuation ':'-'@' → 51-57
         if(ispunct(*word) && '[' <= *word && *word <= '`')
               character = *word - '[' + 58;             // Punctuation '['-'`' → 58-63
         if(ispunct(*word) && '{' <= *word && *word <= '~')
               character = *word - '{' + 64;             // Punctuation '{'-'~' → 64+

         if(!curr->alphaNumeric[character])
               return NULL;                              // If path breaks, word doesn't exist

         curr = curr->alphaNumeric[character];        // Traverse to the next node
         word++;
   }

   return curr; // Return the node (can be used to check if it's a complete word)
}
```

The Search() function looks for a given slang word in the Trie by navigating through its characters one by one. Each character is converted into an index to access child nodes in the alphaNumeric array. If any character doesn't have a corresponding node, it means the word doesn't exist and returns NULL. If all characters match, the function returns the final node, which can then be checked to determine if it marks a valid slang entry (isWord == true) and retrieve its description.

```
void Search_Slang(){

   system("cls");
   // Clear the screen at the start.

   Show_Cursor();
   // Show the console cursor for typing input.

   int i = 0;
   char word[256] = {0};
   // Initialize a character buffer and index for the slang word input.

   while(true){
         // Loop to get user input character by character.

         system("cls");
         Title();
         // Refresh screen and title each time the loop runs.

         Validate_Word(word);
         // Validate the current input.

         printf(" >> Input slang word ['Esc' to return to menu]: %s", word);
         // Show prompt with the current input buffer.

         char ch = _getch();
         // Get a single character from user input without displaying it.

         if(ch == '\e'){
               Hide_Cursor();
               return;
               // If Esc is pressed, exit the function.
         }

         if(ch == '\r'){
```

```
                if(strlen(word) > 1 && !strchr(word, ' ')) break;
                else continue;
                // On Enter, only accept if word is at least 2 characters and contains no space.
            }

            if(ch == '\b' && i > 0) word[--i] = '\0';
            // Handle Backspace: remove last character from input.

            if((isalnum(ch) || ispunct(ch) || isspace(ch)) && i < 255){
                word[i++] = ch;
                word[i] = '\0';
                // Add valid character to the word buffer.
            }
        }

        Hide_Cursor();
        // Hide the cursor after input is done.

        printf("\n\n");
        // Add spacing before showing result.

        node* found = Search(ROOT, word);
        // Search for the slang word in the dictionary (the Trie).

        system("cls");
        Title();
        // Clear the screen and show title again before displaying results.

        if(found && found->isWord){
            printf(" << Slang word  : %s\n", word);
            printf(" << Description : %s\n\n", found->description);
            // If the word is found and marked as valid, display it and its description.
        } else{
            printf(" << The slang \"%s\" is not in the dictionary\n\n", word);
            // If not found, display a not-found message.
        }
```

```
    Press_Any_Key_To_Continue();
    // Wait for the user to press a key before returning to the menu.
}
```

The Search_Slang() function allows the user to search for a specific slang word in the dictionary. It captures the user's input character by character, validates it, and upon pressing Enter, checks if the word exists using the Search() function. If found, it displays the slang and its description; if not, it informs the user. The function is interactive and supports smooth cancellation by pressing Esc.

- Option 3 functions:

```
void Validate_Prefix(char* prefix){

   // Check if the prefix length is more than 1 character
   if(strlen(prefix) > 1) printf(" << Prefix more than 1 character [v]\n");
   else printf(" << Prefix more than 1 character [ ]\n");

   // Check if the prefix contains no spaces
   if(!strchr(prefix, ' ')) printf(" << Prefix contains no space [v]\n\n");
   else printf(" << Prefix contains no space [ ]\n\n");

}
```

The Validate_Prefix() function checks if the user's inputted prefix meets the required criteria to proceed with a prefix-based search in the slang dictionary. It ensures the prefix is more than one character and contains no spaces. It gives visual feedback using checkboxes [v] (valid) or [ ] (invalid) for each condition. This helps guide the user in real-time to input a properly formatted prefix.

```
bool alreadyDigit = false;      // Flag to ensure digit section is printed only once
bool alreadySpecial = false;     // Flag to ensure special character section is printed only
once

void Display(node* curr, char* word, int index, int* num){

   // If current node marks end of a word, print it with numbering
   if(curr->isWord){
        word[index] = '\0';
        printf(" << %d. %s\n", (*num)++, word);
    }

    for(int i = 0; i < ALPHABET_SIZE; i++){
        // At the root, print category headers for letters
        if(curr == ROOT && curr->alphaNumeric[i] && i < 26){
              printf(" << ~ %c ~\n", 'A' + i);
              *num = 1;
        }
        // At the root, print header for digits
        else if(curr == ROOT && curr->alphaNumeric[i] && 26 <= i && i < 36 && !alreadyDigit){
```

```
            printf(" << ~ DIGITS ~\n");
            *num = 1;
            alreadyDigit = true;
        }
        // At the root, print header for special characters
        else if(curr == ROOT && curr->alphaNumeric[i] && 36 <= i && i < 68 && !alreadySpecial){
            printf(" << ~ SPECIAL CHARACTERS ~\n");
            *num = 1;
            alreadySpecial = true;
        }

        // Recursively traverse and build words starting with uppercase letters at root
        if(curr->alphaNumeric[i] && i < 26 && index == 0){
            word[index] = 'A' + i;
            Display(curr->alphaNumeric[i], word, index + 1, num);
        }
        // Lowercase handling for deeper levels
        else if(curr->alphaNumeric[i] && i < 26){
            word[index] = 'a' + i;
            Display(curr->alphaNumeric[i], word, index + 1, num);
        }
        // Digits (0-9)
        if(curr->alphaNumeric[i] && 26 <= i && i < 36){
            word[index] = i + '0' - 26;
            Display(curr->alphaNumeric[i], word, index + 1, num);
        }
        // Special characters group 1 (! to /)
        if(curr->alphaNumeric[i] && 36 <= i && i < 51){
            word[index] = i + '!' - 36;
            Display(curr->alphaNumeric[i], word, index + 1, num);
        }
        // Special characters group 2 (: to @)
        if(curr->alphaNumeric[i] && 51 <= i && i < 58){
            word[index] = i + ':' - 51;
            Display(curr->alphaNumeric[i], word, index + 1, num);
        }
```

```
            // Special characters group 3 ([ to `)
            if(curr->alphaNumeric[i] && 58 <= i && i < 64){
                  word[index] = i + '[' - 58;
                  Display(curr->alphaNumeric[i], word, index + 1, num);
            }
            // Special characters group 4 ({ to ~)
            if(curr->alphaNumeric[i] && 64 <= i && i < 68){
                  word[index] = i + '{' - 64;
                  Display(curr->alphaNumeric[i], word, index + 1, num);
            }

            // Add spacing between groups when returning from root level
            if(curr == ROOT && curr->alphaNumeric[i] && i < 26)
                  printf("\n");
            else if(curr == ROOT && i == 35 && alreadyDigit){
                  printf("\n");
                  alreadyDigit = false;
            }
            else if(curr == ROOT && i == 67 && alreadySpecial){
                  printf("\n");
                  alreadySpecial = false;
            }
      }
   }
```

The Display() function recursively traverses the trie and prints all slang words that start with a specific prefix or are in the dictionary. It classifies words into alphabet letters, digits, and special characters. At the root level, it prints section headers like "A", "DIGITS", or "SPECIAL CHARACTERS", and resets numbering for each section. For each completed word, it prints it with numbering and builds the word dynamically during traversal. This function helps in listing dictionary contents neatly and in organized sections for better user readability.

```
void View_With_Prefix(){
    // Function to display slang words from a dictionary that match a user-given prefix.

      system("cls");
    // Clears the console screen.
```

```
  Title();
// Displays the title/header of the program (assumed to be defined elsewhere).

  Show_Cursor();
// Makes the cursor visible, usually for text input.

  int i = 0;
// Index to track the length of user input (prefix).

  char prefix[256] = {0};
// Buffer to store the user's prefix input, initialized to zero.

  while(true){
     // Infinite loop to keep accepting input until user confirms or exits.

        system("cls");
     // Clears the screen each loop iteration to refresh the view.

        Title();
     // Re-displays the title after clearing the screen.

        Validate_Prefix(prefix);
     // Optional function call to validate or perhaps format the prefix (defined elsewhere).

        printf(" >> Input the prefix ['Esc' to return to menu]: %s", prefix);
     // Displays current input and instruction to user.

        char ch = _getch();
     // Gets a character from user input without waiting for enter and without echoing.

        if(ch == '\e'){
         // If the escape key is pressed...
              Hide_Cursor();
         // Hides the cursor before exiting.
              return;
```

```
            // Exit the function (return to menu).
        }

        if(ch == '\r'){
            // If Enter is pressed...
                if(strlen(prefix) > 1 && !strchr(prefix, ' ')) break;
            // Only proceed if prefix has more than 1 character and contains no space.
                else continue;
            // Otherwise, ignore and keep looping.
        }

        if(ch == '\b' && i > 0) prefix[--i] = '\0';
    // If Backspace is pressed and there's something to delete, remove last character.

        if((isalnum(ch) || ispunct(ch) || isspace(ch)) && i < 255){
            // If the input is alphanumeric, punctuation, or space, and there's room...
                prefix[i++] = ch;
            // Add character to prefix buffer.
                prefix[i] = '\0';
            // Null-terminate the string.
        }
    }

    Hide_Cursor();
     // Hide the cursor after input is complete.

    printf("\n\n");
     // Adds spacing before the result output.

    int num = 1;
     // Initialize a counter for display numbering.

    char word[256] = {0};
     // Buffer to store a copy of the prefix for search purposes.

    strcpy(word, prefix);
```

```
        // Copy the prefix into 'word'.

      node* found = Search(ROOT, word);
        // Searches the dictionary tree (starting from ROOT) for entries matching the prefix.

      system("cls");
        // Clears screen again before displaying results.

      Title();
        // Displays the title again.

      if(found){
            // If a matching node is found...
            printf(" << Slang found in the dictionary with prefix \"%s\" :\n", prefix);
            // Inform the user of a successful match.
            Display(found, word, strlen(prefix), &num);
            // Display all matching slang words, passing prefix length and number counter.
      }
      else printf(" << The slang with the prefix \"%s\" is not in the dictionary\n", prefix);
        // If not found, display a message.

      printf("\n");
        // Add space after results.

      Press_Any_Key_To_Continue();
        // Wait for user input before continuing (pause screen).
}
```

The View_With_Prefix() function allows users to search for and display all slang words in the dictionary that begin with a specific prefix. It takes user input character by character, validates it, and once a valid prefix is entered, it searches the dictionary for matches and displays them. If no matches are found, it notifies the user accordingly.

- Option 4 functions:

```
void View_All(){

   system("cls");  // Clear the console screen

   Title();  // Display the title

   bool thereIsWord = false;

   // Check if the Trie has at least one word
   for(int i = 0; i < ALPHABET_SIZE; i++){
         if(ROOT->alphaNumeric[i]){
               thereIsWord = true;
               break;
         }
   }

   if(thereIsWord){
         printf(" << Dictionary :\n\n");

         int num = 1;                // Counter for numbering the words
         char word[256] = {0};       // Buffer to build words while traversing

         Display(ROOT, word, 0, &num);  // Call recursive function to print all words
   } else{
         // If dictionary is empty
         printf(" << There is no word in the dictionary\n\n");
   }

   Press_Any_Key_To_Continue();  // Wait for user input before returning

}
```

The View_All() function shows all slang words stored in the dictionary (Trie). It first checks whether any words exist in the Trie. If the Trie is empty, it displays a message accordingly. Otherwise, it calls the recursive Display() function to print each word, organized by sections (letters, digits, special characters), along with a numbered list.

- Option 5 functions:

```
void Exit(){

   system("cls");    // Clear the console screen
   Title();          // Display the title

   printf(" << ");
   const char* message1 = "Until next time sigma,\n";
   for(int i = 0; message1[i] != '\0'; i++){
         printf("%c", message1[i]); // Print message1 character by character
         Sleep(50);                 // Add delay for dramatic effect
   }

   printf(" << ");
   const char* message2 = "Stay skibidi ohio and be a hydrated rizzler!";
   for(int i = 0; message2[i] != '\0'; i++){
         printf("%c", message2[i]); // Print message2 character by character
         Sleep(50);                 // Delay again for style
   }

   Sleep(1500);        // Wait a bit longer before exiting
   system("cls");      // Clear the screen one last time

   exit(0);            // Exit the program successfully

}
```

The Exit() function gracefully ends the program. It displays a humorous farewell message character-by-character with delays for animation-like output. After a short pause, it clears the screen and calls exit(0) to terminate the application. This adds personality and a fun user experience when leaving the program.

CUSTOM CASE

- Input of 15 slang words:

```
=======================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                              GG   LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                GG   GG
                                 GGGGG

=======================================================

<< Slang word: "skibidibrainrot"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A chaotic state of mind caused by overexposure to memes and TikTok.
```

```
=====================================================

 BBBBB                                      LL
BB   BB                                     LL
BB   BB                                 GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "skibidisigma"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A stoic, cold individual who radiates chaotic energy.
```

```
=========================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=========================================================

<< Slang word: "skibidify"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: The act of becoming unhinged, often paired with erratic movement or noi
ses.
```

```
=====================================================

 BBBBB                                      LL
BB   BB                                     LL
BB   BB                                 GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word: "skibidispeedrun"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: Rapid descent into madness, done in record time.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                                GG LL
BB  BB      OOOOO      OOOOO      GGGGG  LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG  LL    EEEEE
                                    GG
                                    GG
                                 GG   GG
                                  GGGGG

=====================================================

<< Slang word: "skibidiboom"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: An explosion of random behavior, often with no context.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "gyattalert"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: Internal alarm triggered by exaggerated meme-worthy curves.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                              GG   LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word: "rizzquake"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A flirty move so intense it shakes your social circle.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "goblinmode"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: Living feral and free, no shame, no plan.
```

```
=====================================================

 BBBBB                                      LL
BB   BB                                     LL
BB   BB                                 GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "sigmaslopper"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A mysterious alpha who eats like a beast yet stays cool.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                                GG LL
BB  BB      OOOOO      OOOOO      GGGGG  LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG  LL    EEEEE
                                    GG
                                    GG
                                 GG   GG
                                  GGGGG

=====================================================

<< Slang word: "ohiofinalboss"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: The ultimate cursed being from the depths of meme-Ohio.
```

```
=====================================================

 BBBBB                                  LL
BB   BB                                 LL
BB   BB                             GG  LL
BB  BB      OOOOO      OOOOO      GGGGG  LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG  LL    EEEEE
                                    GG
                                    GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word: "gloopgobbler"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: One who devours all slime, both metaphorically and literally.
```

```
=====================================================

 BBBBB                                  LL
BB   BB                                 LL
BB   BB                             GG  LL
BB  BB      OOOOO      OOOOO      GGGGG  LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG  LL    EEEEE
                                    GG
                                    GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word: "meowfication"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: The transformation into a chaotic, attention-seeking feline.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "deluluwarrior"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A soldier for the delusional army, armed with false hope.
```

```
=====================================================

 BBBBB                                      LL
BB   BB                                     LL
BB   BB                                 GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word: "npcrizzy"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: An oddly robotic yet surprisingly smooth flirt.
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                              GG   LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word: "rizzlerbeast9000"

<< Description more than 1 word [v]

>> Input description ['Esc' to return to menu]: A futuristic menace of unstoppable flirtation energy.
```

- Search 5 words:

```
=====================================================

 BBBBB                                      LL
BB   BB                                     LL
BB   BB                                 GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                    GG
                                    GG
                                GG   GG
                                 GGGGG

=====================================================

<< Slang word  : Deluluwarrior
<< Description : A soldier for the delusional army, armed with false hope.

>> Press any key to continue...
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word  : Ohiofinalboss
<< Description : The ultimate cursed being from the depths of meme-Ohio.

>> Press any key to continue...
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word  : Gyattalert
<< Description : Internal alarm triggered by exaggerated meme-worthy curves.

>> Press any key to continue...
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word  : Rizzlerbeast9000
<< Description : A futuristic menace of unstoppable flirtation energy.

>> Press any key to continue...
```

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang word  : NPCrizzy
<< Description : An oddly robotic yet surprisingly smooth flirt.

>> Press any key to continue...
```

- View prefix 5 words:

```
=====================================================

 BBBBB                                    LL
BB   BB                                   LL
BB   BB                               GG  LL
BB  BB      OOOOO      OOOOO      GGGGG   LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG  LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG  LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG   LL    EEEEE
                                     GG
                                     GG
                                  GG   GG
                                   GGGGG

=====================================================

<< Slang found in the dictionary with prefix "SKIBIDI" :
<< 1. SKIBIDIboom
<< 2. SKIBIDIbrainrot
<< 3. SKIBIDIfy
<< 4. SKIBIDIsigma
<< 5. SKIBIDIspeedrun

>> Press any key to continue...
```

- View all:

```
=====================================================

 BBBBB                                   LL
BB   BB                                  LL
BB   BB                              GG  LL
BB  BB      OOOOO      OOOOO      GGGGG  LL    EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE   EE
BB    BB  OO     OO  OO     OO  GG    GG LL  EE  EEEEE
BB   BB    OO   OO    OO   OO    GG   GG LL   EE
 BBBBB      OOOOO      OOOOO      GGGGG  LL    EEEEE
                                     GG
                                     GG
                                 GG   GG
                                  GGGGG

=====================================================

<< Dictionary :

<< ~ D ~
<< 1. Deluluwarrior

<< ~ G ~
<< 1. Gloopgobbler
<< 2. Goblinmode
<< 3. Gyattalert

<< ~ M ~
<< 1. Meowfication

<< ~ N ~
<< 1. Npcrizzy

<< ~ O ~
<< 1. Ohiofinalboss

<< ~ R ~
<< 1. Rizzlerbeast9000
<< 2. Rizzquake

<< ~ S ~
<< 1. Sigmaslopper
<< 2. Skibidiboom
<< 3. Skibidibrainrot
<< 4. Skibidify
<< 5. Skibidisigma
<< 6. Skibidispeedrun

>> Press any key to continue...
```

Soal

Case

Boogle

Boogle is a company that create, and document new slang words based on the internet. You as a programmer working at the Boogle company are asked to create an application that is useful for seeing what slang words have been released by Boogle. **Ensure that you didn't use regex in your work, or it will affect your score. Please focus on the main logic and main feature! (Design are not scored)**. The requirements are:

- The application consists of **5 menus**:
  1. **Release a new slang word**
  2. **Search a slang word**
  3. **View all slang words starting with a certain prefix word**
  4. **View all slang words**
  5. **Exit**

- If user choose menu **1** ("**Release a new slang word**"), then the program will:
  - Ask the user to **input** the **new slang word**. Validate that the **slang word** must be **more than 1 character and contains no space**.
  - Ask the user to **input** the **description (meaning)** of the new word. Validate that the **description** must be **more than 1 word**.
  - **Store** the new released slang word to a **Trie** data structure along with its description.

```
Input a new slang word [Must be more than 1 characters and contains no space]: d
Input a new slang word [Must be more than 1 characters and contains no space]: d a
Input a new slang word [Must be more than 1 characters and contains no space]: da
Input a new slang word description [Must be more than 2 words]: The
Input a new slang word description [Must be more than 2 words]: The word the

Successfully released new slang word.
Press enter to continue...
```

- If the slang word **already exists** in the Trie, then **update** the description with the new description.

```
Input a new slang word [Must be more than 1 characters and contains no space]: da
Input a new slang word description [Must be more than 2 words]: Same meaning as word “the”

Successfully updated a slang word.
Press enter to continue...
```

- If user choose menu **2** (“**Search a slang word**”), then the program will:
  - Ask the user to **input** the **slang word that want to be searched**. Validate that the **slang word** must be **more than 1 character and contains no space**.
  - **Search** the input word in the Trie data structure.
  - If there **is no such word**, please **show empty message** for the user and go back to main menu.

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: de

There is no word “de” in the dictionary.
Press enter to continue...
```

- If there **is such word**, please **show the word along with its description**.

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: d
Input a slang word to be searched [Must be more than 1 characters and contains no space]: d a
Input a slang word to be searched [Must be more than 1 characters and contains no space]: da

Slang word  : da
Description : Same meaning as word “the”

Press enter to continue...
```

- If user choose menu **3** (“**View all slang words starting with a certain prefix word**”), then the program will:
  - Ask the user to **input** the **prefix word that want to be searched**.
  - **Search** the input word in the Trie data structure.
  - If there **is no such word**, please **show empty message** for the user.

```
Input a prefix to be searched: de

There is no prefix “de” in the dictionary.
Press enter to continue...
```

  - If there **is such word**, please **show the list of words in the dictionary that starts with the prefix word in lexicographical order.**

```
Input a prefix to be searched: da

Words starts with “da”:
1. da
2. dadan
3. daijobu
4. dawg
5. dazz

Press enter to continue...
```

- If user choose menu **4** (“**View all slang words**”), then the program will:
  - If there **is no word yet** in the dictionary, please **show empty message** for the user.

```
There is no slang word yet in the dictionary.
Press enter to continue...
```

  - Else, please **show the list of all words in the dictionary** in **lexicographical order.**

```
List of all slang words in the dictionary:
1. da
2. dadan
3. daijobu
4. dawg
5. dazz
6. rizz
7. simp

Press enter to continue...
```

- If user choose menu **5** ("**Exit**"), then the program will be closed.

```
Thank you... Have a nice day :)
```